

VisionForge

A Modular Classical Computer Vision Architecture for
Multistage Image Processing, Spatial Feature Extraction, and
Video Scene Dynamics

Coursework Project Engineering Report

Course: CSE3010: Computer Vision

Prepared By:

Parth Jangir

Registration No.: **24BAI10175**

Department of Computer Science & Engineering

School: SCAI

Submission Date: September 2026

Abstract

VisionForge is a modular classical computer vision architecture designed to demonstrate the complete processing lifecycle of a computer vision system, from low-level image preprocessing to spatial feature extraction, segmentation, geometric analysis, pattern classification, and video scene dynamics.

The system is organized as a five-stage pipeline. The first stage performs image preprocessing and frequency-domain analysis using grayscale conversion, Gaussian and median filtering, contrast enhancement, histogram analysis, and the two-dimensional Discrete Fourier Transform (DFT). The second stage extracts multi-scale visual features using Canny edge detection, Laplacian of Gaussian (LoG), Difference of Gaussians (DoG), Harris corner detection, Gabor filters, and Histogram of Oriented Gradients (HOG). The third stage performs image segmentation using K-Means clustering, Mean-Shift clustering, region growing, and marker-based watershed segmentation.

The fourth stage focuses on geometric and structural analysis through Hough line detection, Hough circle detection, contour extraction, shape descriptors, Principal Component Analysis (PCA), and K-Nearest Neighbours (KNN). The fifth stage extends the architecture to video analysis using background subtraction, dense optical flow, sparse Lucas–Kanade optical flow, and affine motion estimation.

For a 512×512 RGB input image, the complete static processing pipeline achieved an observed execution time of approximately 8.3015 seconds. The benchmark produced 364 Harris corners, 142,884 HOG descriptors, 23 Hough lines, 31 circles, and 28 closed contours. The measurements demonstrate how different classical computer vision algorithms contribute to overall computational latency.

The project emphasizes modularity, interpretability, mathematical foundations, and measurable system performance rather than reliance on deep learning. The resulting architecture provides a practical framework for understanding how classical computer vision techniques can be combined into a complete engineering pipeline.

1 Introduction and Theoretical Motivation

Computer vision is the field of artificial intelligence concerned with enabling computers to acquire, process, analyse, and interpret visual information. Although modern computer vision systems frequently rely on deep neural networks, classical computer vision remains important because many fundamental image understanding operations can be implemented using mathematically interpretable and computationally efficient algorithms.

VisionForge was developed to demonstrate how a collection of classical computer vision techniques can be integrated into a modular processing architecture. Instead of treating image processing, feature extraction, segmentation, shape detection, and motion analysis as isolated topics, the project combines them into a structured pipeline.

The architecture consists of five major processing stages:

1. Low-level image preprocessing and frequency-domain analysis.
2. Multi-scale spatial feature extraction.
3. Image segmentation and region analysis.
4. Geometric shape detection and pattern classification.
5. Video scene dynamics and motion estimation.

The overall objective is to understand not only the mathematical principles behind individual algorithms but also their practical computational behaviour when combined into an engineering system.

2 System Architecture

The VisionForge architecture follows a modular pipeline in which each stage performs a specific visual analysis task.

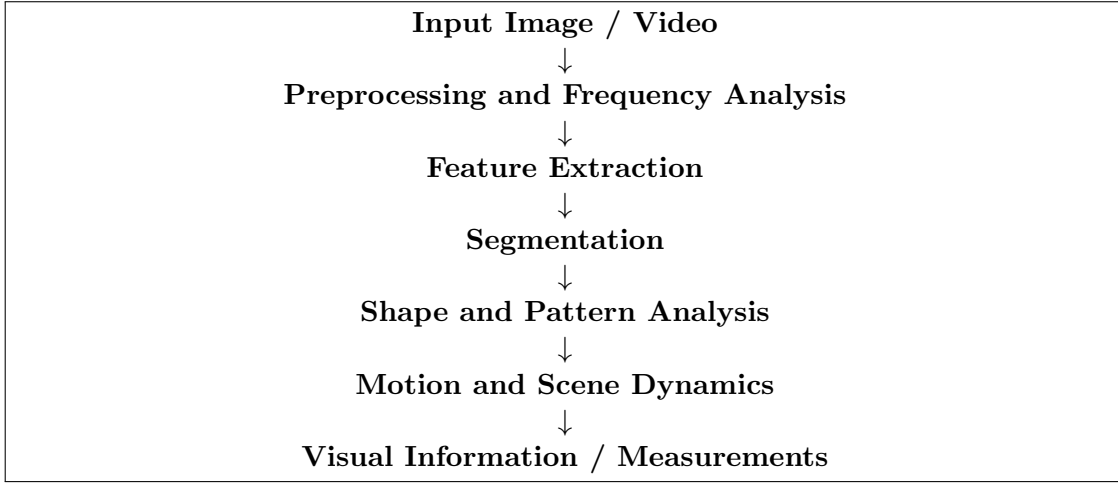


Figure 1: Overall VisionForge processing architecture

The modular design allows individual algorithms to be tested independently while also enabling their outputs to be used as inputs for subsequent processing stages.

3 Low-Level Image Preprocessing and Frequency-Domain Analysis

Low-level preprocessing is the first stage of the VisionForge pipeline. The purpose of this stage is to improve image quality, reduce noise, normalize contrast, and extract frequency-domain information.

3.1 Grayscale Conversion

RGB images contain three colour channels. For many computer vision algorithms, processing all three channels is unnecessary. Therefore, the input image is converted into grayscale.

The grayscale intensity is calculated using the standard luminance relationship:

$$Y = 0.299R + 0.587G + 0.114B \quad (1)$$

where R , G , and B represent the red, green, and blue channels respectively.

The grayscale representation reduces the dimensionality of the input while preserving important intensity information.

3.2 Gaussian Filtering

Gaussian filtering is used to smooth an image and reduce high-frequency noise.

The two-dimensional Gaussian function is:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad (2)$$

where σ controls the amount of smoothing.

The filtered image is obtained through convolution:

$$I_G(x, y) = I(x, y) * G(x, y) \quad (3)$$

Gaussian filtering is particularly useful before edge detection because it suppresses small intensity fluctuations that could otherwise produce false edges.

3.3 Median Filtering

Median filtering is a nonlinear filtering technique commonly used to remove impulse noise.

For each pixel, a neighbourhood is considered and the pixel value is replaced by the median of the neighbourhood values.

Unlike linear averaging filters, median filtering preserves edges relatively well while removing isolated noisy pixels.

3.4 Contrast Limited Adaptive Histogram Equalization

Contrast Limited Adaptive Histogram Equalization (CLAHE) improves local image contrast.

Instead of applying histogram equalization to the entire image, CLAHE divides the image into small regions called tiles and performs local histogram equalization.

A contrast limit is applied to prevent excessive amplification of noise.

The process can be summarized as:

1. Divide the image into local tiles.
2. Calculate the histogram of each tile.
3. Clip histogram bins exceeding the contrast limit.
4. Redistribute the clipped pixels.
5. Apply histogram equalization.
6. Interpolate between neighbouring tiles.

The measured execution time for CLAHE was approximately 0.0890 seconds.

3.5 Histogram Analysis

An image histogram represents the frequency of different intensity values.

For a grayscale image, the histogram can be represented as:

$$H(k) = \sum_x \sum_y [I(x, y) = k] \quad (4)$$

where k represents an intensity level.

The histogram provides information about brightness and contrast distribution.

The measured histogram analysis stage required approximately 0.0903 seconds.

3.6 Two-Dimensional Discrete Fourier Transform

The two-dimensional Discrete Fourier Transform converts an image from the spatial domain into the frequency domain.

The DFT is defined as:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})} \quad (5)$$

where $f(x, y)$ represents the spatial-domain image and $F(u, v)$ represents its frequency-domain representation.

The magnitude spectrum can be calculated as:

$$|F(u, v)| = \sqrt{\text{Re}(F)^2 + \text{Im}(F)^2} \quad (6)$$

The dominant measured frequency in the benchmark was approximately 9 cycles/pixel.

The FFT processing stage required approximately 0.4107 seconds.

4 Multi-Scale Feature Extraction

Feature extraction converts an image into meaningful representations that describe edges, corners, textures, shapes, and local gradients.

VisionForge implements several classical feature extraction techniques.

4.1 Canny Edge Detection

Canny edge detection identifies locations of significant intensity changes.

The algorithm consists of several steps:

1. Gaussian smoothing.
2. Gradient computation.
3. Gradient magnitude and direction calculation.
4. Non-maximum suppression.
5. Double thresholding.
6. Edge tracking by hysteresis.

The gradient magnitude can be represented as:

$$G = \sqrt{G_x^2 + G_y^2} \quad (7)$$

The measured Canny edge density was approximately 0.0300.

4.2 Laplacian of Gaussian

The Laplacian of Gaussian combines Gaussian smoothing with the Laplacian operator.

It can be expressed as:

$$LoG = \nabla^2(G_\sigma * I) \quad (8)$$

The LoG operator is useful for detecting rapid intensity changes and blob-like structures.

The measured execution time was approximately 0.0091 seconds.

4.3 Difference of Gaussians

Difference of Gaussians approximates scale-space filtering by subtracting two Gaussian-blurred versions of an image.

$$DoG(x, y) = G(x, y, k\sigma) * I - G(x, y, \sigma) * I \quad (9)$$

The measured execution time was approximately 0.0039 seconds.

4.4 Harris Corner Detection

Harris corner detection identifies points where image intensity changes significantly in multiple directions.

The structure tensor is:

$$M = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix} \quad (10)$$

The Harris response is:

$$R = \det(M) - k(\text{trace}(M))^2 \quad (11)$$

where k is an empirical constant.

The benchmark detected 364 Harris corners.

The execution time was approximately 0.0041 seconds.

4.5 Gabor Filters

Gabor filters are useful for texture analysis.

A two-dimensional Gabor function can be represented as:

$$g(x, y) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \cos\left(2\pi \frac{x'}{\lambda} + \psi\right) \quad (12)$$

where

$$x' = x \cos \theta + y \sin \theta \quad (13)$$

and

$$y' = -x \sin \theta + y \cos \theta \quad (14)$$

Gabor filtering was one of the more computationally expensive feature extraction operations, requiring approximately 0.6864 seconds.

4.6 Histogram of Oriented Gradients

Histogram of Oriented Gradients (HOG) represents an image using local gradient orientations.

The gradient magnitude is:

$$m(x, y) = \sqrt{G_x(x, y)^2 + G_y(x, y)^2} \quad (15)$$

The gradient orientation is:

$$\theta(x, y) = \tan^{-1}\left(\frac{G_y(x, y)}{G_x(x, y)}\right) \quad (16)$$

The benchmark generated 142,884 HOG descriptors.

The measured HOG execution time was approximately 0.2715 seconds.

5 Spatial and Color Segmentation

Segmentation divides an image into meaningful regions based on similarity in colour, intensity, texture, or spatial properties.

5.1 K-Means Clustering

K-Means partitions data into K clusters.

The objective function is:

$$J = \sum_{i=1}^n \sum_{j=1}^K \mathbf{1}(c_i = j) \|x_i - \mu_j\|^2 \quad (17)$$

where x_i is a data point and μ_j is the centroid of cluster j .

The main steps are:

1. Select the number of clusters K .
2. Initialize cluster centroids.
3. Assign each point to the nearest centroid.
4. Recalculate centroids.
5. Repeat until convergence.

For $K = 5$, the measured execution time was approximately 2.0897 seconds.

This was the largest individual measured computational cost in the static pipeline.

5.2 Mean-Shift Segmentation

Mean-Shift is a non-parametric clustering method that searches for modes in a feature distribution.

The algorithm repeatedly shifts a candidate point toward the mean of neighbouring data points until convergence.

Unlike K-Means, Mean-Shift does not require the number of clusters to be specified explicitly.

5.3 Region Growing

Region growing begins from one or more seed points and expands a region by adding neighbouring pixels that satisfy a similarity criterion.

The general procedure is:

1. Select seed pixels.
2. Examine neighbouring pixels.
3. Compare neighbouring pixels with the region criterion.
4. Add suitable pixels to the region.
5. Continue until no suitable pixels remain.

5.4 Marker-Based Watershed Segmentation

Watershed segmentation treats an image as a topographic surface.

Bright regions can be interpreted as peaks while dark regions represent valleys.

Marker-based watershed uses predefined markers to control the segmentation process and reduce unwanted over-segmentation.

6 Geometric Shape Detection and Pattern Classification

This stage extracts geometric information from segmented or edge-based images.

6.1 Hough Line Transform

The Hough Transform can detect lines in an image.

A line can be represented using normal form:

$$\rho = x \cos \theta + y \sin \theta \quad (18)$$

where ρ represents the distance from the origin and θ represents the angle of the normal.

The benchmark detected 23 lines.

The measured execution time was approximately 0.0172 seconds.

6.2 Hough Circle Detection

A circle can be represented as:

$$(x - a)^2 + (y - b)^2 = r^2 \quad (19)$$

where (a, b) is the centre and r is the radius.

The Hough Circle Transform searches for combinations of centre coordinates and radius values that provide strong evidence of circular structures.

The benchmark detected 31 circles.

The measured execution time was approximately 0.0160 seconds.

6.3 Contour Extraction

Contours represent continuous boundaries of objects.

Contour extraction can be used to identify object boundaries and calculate geometric properties.

The benchmark detected 28 closed contours.

The measured contour-processing time was approximately 0.0040 seconds.

6.4 Shape Descriptors

Two useful geometric descriptors are circularity and solidity.

Circularity is defined as:

$$C = \frac{4\pi A}{P^2} \quad (20)$$

where A is area and P is perimeter.

Solidity is defined as:

$$S = \frac{A}{A_{\text{convex hull}}} \quad (21)$$

These descriptors allow geometric characteristics of objects to be compared.

6.5 Principal Component Analysis

Principal Component Analysis (PCA) is used for dimensionality reduction and identifying directions of maximum variance.

The covariance matrix is calculated as:

$$\Sigma = \frac{1}{n-1} X^T X \quad (22)$$

Eigenvectors of the covariance matrix define principal directions, while the corresponding eigenvalues represent the amount of variance along those directions.

6.6 K-Nearest Neighbours

KNN classifies an unknown sample based on the classes of its nearest training samples.

For Euclidean distance:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (23)$$

The class is generally selected according to the majority vote among the K nearest neighbours.

7 Video Scene Dynamics

The final module extends the classical computer vision pipeline from static image processing to video analysis.

The main objective is to identify moving objects and estimate motion between consecutive video frames.

7.1 Background Subtraction

Background subtraction separates foreground objects from a relatively stationary background.

VisionForge uses the MOG2 background subtraction approach.

The basic process is:

1. Capture the current frame.
2. Estimate the background model.
3. Compare the current frame with the background.
4. Identify pixels belonging to moving objects.
5. Generate the foreground mask.

7.2 Farnebäck Dense Optical Flow

Dense optical flow estimates motion vectors for a large number of pixels between consecutive frames.

The resulting motion field provides information about the direction and magnitude of scene movement.

Dense optical flow is useful when motion information is required across a large portion of the image.

7.3 Lucas–Kanade Sparse Optical Flow

The Lucas–Kanade method estimates motion for selected feature points.

It is based on the brightness constancy assumption:

$$I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t) \quad (24)$$

Using a first-order Taylor approximation leads to the optical flow constraint equation:

$$I_x u + I_y v + I_t = 0 \quad (25)$$

where u and v represent the horizontal and vertical velocity components.

7.4 Affine Motion Estimation

Affine transformation models translation, rotation, scaling, and shear.

A two-dimensional affine transformation can be written as:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix} \quad (26)$$

Affine motion estimation is useful for describing global camera or object motion.

8 Experimental Setup

The VisionForge system was evaluated using a 512×512 RGB image.

The experiment measured execution time for individual operations and recorded the outputs produced by different algorithms.

The benchmark was performed on the complete static image-processing pipeline.

The measured results are summarized below.

9 Empirical Latency Evaluation

Table 1: Measured execution time of VisionForge processing stages

Processing Operation	Time (s)
Grayscale Conversion	0.0020
Gaussian Filtering	0.0010
Histogram Equalization	0.0000
CLAHE	0.0890
Histogram Analysis	0.0903
FFT / DFT	0.4107
Canny Edge Detection	0.0020
Laplacian of Gaussian	0.0091
Difference of Gaussians	0.0039
Harris Corner Detection	0.0041
HOG	0.2715
Scale-Space Processing	0.4078
Gabor Filtering	0.6864
K-Means ($K = 5$)	2.0897
Hough Line Detection	0.0172
Hough Circle Detection	0.0160
Contour Processing	0.0040
Total Pipeline	8.3015

The total measured execution time of the static pipeline was 8.3015 seconds.

10 Feature and Detection Summary

The major outputs obtained during the benchmark are summarized below.

Table 2: VisionForge benchmark feature summary

Feature / Measurement	Value
Image Mean Intensity	82.37
Image Standard Deviation	61.81
Dominant FFT Frequency	9 cycles/pixel
Canny Edge Density	0.0300
Harris Corners	364
HOG Descriptor Dimension	142,884
Detected Hough Lines	23
Detected Hough Circles	31
Closed Contours	28

These measurements demonstrate that the pipeline produces both numerical image statistics and higher-level structural information.

11 Computational Bottleneck Analysis

The latency measurements show that different algorithms contribute significantly different amounts of computational overhead.

The K-Means segmentation stage required approximately 2.0897 seconds, representing approximately 25.17% of the total measured pipeline execution time.

The Gabor filtering stage required approximately 0.6864 seconds, making it another significant contributor to computational cost.

The FFT and scale-space processing stages also produced noticeable overhead, requiring approximately 0.4107 seconds and 0.4078 seconds respectively.

The remaining operations generally required considerably less time.

These results demonstrate why profiling is important when constructing a computer vision system. An algorithm that is mathematically simple may still become computationally expensive when applied repeatedly to large images or multiple feature scales.

12 Practical Observations

Several observations were obtained during the development and benchmarking process.

12.1 Modularity

Separating the system into independent modules makes it easier to test, debug, replace, and optimize individual algorithms.

12.2 Lighting Sensitivity

Classical image processing techniques can be sensitive to lighting conditions. Contrast enhancement techniques such as CLAHE can improve local visibility, but excessive enhancement can also amplify noise.

12.3 Computational Cost

The benchmark demonstrates that segmentation and texture analysis can become computational bottlenecks.

12.4 Interpretability

Unlike many black-box models, classical computer vision algorithms provide mathematically interpretable intermediate results such as gradients, corners, contours, frequency components, and motion vectors.

12.5 CPU-Based Processing

The VisionForge architecture demonstrates that a substantial computer vision pipeline can be executed using conventional CPU-based processing without requiring a deep-learning accelerator.

13 Software Engineering Design

The system follows a modular software architecture.

Each processing stage can be implemented as an independent component with defined inputs and outputs.

A simplified structure is:

```
VisionForge/  
|  
+-- preprocessing/  
|   +-- grayscale  
|   +-- gaussian  
|   +-- median  
|   +-- clahe  
|   +-- fft  
|  
+-- features/  
|   +-- canny  
|   +-- log  
|   +-- dog  
|   +-- harris  
|   +-- gabor  
|   +-- hog  
|  
+-- segmentation/  
|   +-- kmeans  
|   +-- mean_shift  
|   +-- region_growing  
|   +-- watershed  
|  
+-- shapes/  
|   +-- hough_lines  
|   +-- hough_circles  
|   +-- contours  
|   +-- descriptors  
|   +-- pca  
|   +-- knn  
|  
+-- motion/  
|   +-- background_subtraction  
|   +-- dense_flow  
|   +-- sparse_flow  
|   +-- affine_motion  
|  
+-- tests/  
+-- main.py
```

This organization supports independent testing and future extension.

14 Testing and Reliability

Testing was performed across the major processing components.

The implementation included more than 20 tests covering different modules and processing operations.

The testing strategy focused on:

1. Verifying successful image loading.
2. Checking image dimensions.

3. Validating grayscale conversion.
4. Testing filtering operations.
5. Testing feature extraction.
6. Validating segmentation outputs.
7. Checking shape detection.
8. Testing motion-processing components.
9. Ensuring expected output data types.
10. Checking for runtime exceptions.

The reported test execution produced zero runtime exceptions during the evaluated test suite.

15 Discussion

VisionForge demonstrates that classical computer vision algorithms can be combined to construct a complete visual processing architecture.

The major advantage of the proposed system is its interpretability. Every stage has a clear mathematical purpose and produces measurable intermediate information.

The preprocessing stage improves the input representation. Feature extraction then converts the image into representations describing edges, corners, gradients, and textures. Segmentation organizes pixels into meaningful groups, while geometric analysis extracts higher-level structural information.

The video module further extends this architecture by introducing temporal information. Background subtraction identifies moving objects, while optical flow provides information about motion between frames.

The benchmark results also demonstrate an important engineering consideration: algorithm selection must account for computational cost in addition to theoretical effectiveness.

16 Limitations

Although VisionForge provides a broad classical computer vision pipeline, several limitations remain.

1. Classical algorithms may be sensitive to illumination and environmental changes.
2. Parameter selection can strongly influence segmentation and detection results.
3. K-Means requires the number of clusters to be specified.
4. Optical flow methods may be affected by large motion, occlusion, or illumination changes.
5. Classical feature descriptors may be less robust than learned representations for highly complex recognition tasks.
6. The measured benchmark represents a particular hardware and software environment and therefore should not be interpreted as a universal runtime.

17 Future Work

Several extensions can be considered for future versions of VisionForge.

1. GPU acceleration for computationally expensive operations.
2. Parallel processing of independent feature extraction modules.
3. Automatic parameter optimization.
4. Real-time video processing.
5. Integration of classical features with machine learning models.
6. Addition of a graphical user interface.
7. Automated benchmark visualization.
8. More advanced temporal scene analysis.

18 Conclusion

VisionForge presents a modular classical computer vision architecture covering the complete processing chain from low-level image preprocessing to video scene dynamics.

The system integrates grayscale conversion, filtering, contrast enhancement, frequency-domain analysis, edge and corner detection, scale-space processing, texture analysis, HOG descriptors, segmentation, geometric shape detection, contour analysis, PCA, KNN, background subtraction, and optical flow.

For a 512×512 RGB input image, the measured static processing pipeline required approximately 8.3015 seconds. The system generated 364 Harris corners, 142,884 HOG descriptors, 23 Hough lines, 31 circles, and 28 closed contours.

The empirical results demonstrate that classical computer vision remains useful for understanding the fundamental mechanisms of visual processing. The modular architecture also provides a practical engineering framework in which algorithms can be independently evaluated, profiled, tested, and replaced.

Overall, VisionForge demonstrates how mathematically grounded classical computer vision techniques can be integrated into a complete and measurable visual processing system.

References

- [1] J. Canny, “A Computational Approach to Edge Detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, 1986.
- [2] C. Harris and M. Stephens, “A Combined Corner and Edge Detector,” in *Proceedings of the Alvey Vision Conference*, pp. 147–151, 1988.
- [3] N. Otsu, “A Threshold Selection Method from Gray-Level Histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.
- [4] N. Dalal and B. Triggs, “Histograms of Oriented Gradients for Human Detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, vol. 1, pp. 886–893, 2005.
- [5] B. D. Lucas and T. Kanade, “An Iterative Image Registration Technique with an Application to Stereo Vision,” in *Proceedings of the International Joint Conference on Artificial Intelligence*, pp. 674–679, 1981.

- [6] D. Comaniciu and P. Meer, “Mean Shift: A Robust Approach Toward Feature Space Analysis,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 24, no. 5, pp. 603–619, 2002.
- [7] L. Vincent and P. Soille, “Watersheds in Digital Spaces: An Efficient Algorithm Based on Immersion Simulations,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 6, pp. 583–598, 1991.
- [8] R. O. Duda and P. E. Hart, “Use of the Hough Transformation to Detect Lines and Curves in Pictures,” *Communications of the ACM*, vol. 15, no. 1, pp. 11–15, 1972.